Multimodal Fusion

Reproducible Image–Text Classification

A controlled synthetic benchmark and technical evaluation

Technical Project Report

October 2026

Dataset: 1,800 generated paired samples | 24×24 images | 10 text features

Contents

1 Executive summary **2**

2 Research objective and experimental design **2**

2.1 Why the two modalities are complementary 2

3 Model architecture **2**

4 Evaluation protocol and metrics **2**

5 Learning dynamics **4**

6 Confusion analysis **4**

7 Latent representations **6**

8 Interpretation and limitations **6**

9 Reproducibility and repository contents **6**

1 Executive summary

This study tests whether joint learning from a noisy image and a noisy short text representation improves binary classification in a reproducible synthetic setting. Three independently trained networks share a fixed dataset partition. The observed held-out accuracies are **80.00%** (image only), **77.11%** (text only), and **100.00%** (fusion). Zeroing the text representation at inference in the trained fusion model produces **47.78%** accuracy. These numbers were computed by the accompanying executable and are not copied from the source proposal.

The synthetic task is deliberately constructed for cross-modal complementarity. Consequently, the results do not establish performance on natural-language, medical, or real-world images.

2 Research objective and experimental design

Each sample is (X_i, T_i, y_i) with an image $X_i \in [0,1]^{1\times 24\times 24}$, a ten-dimensional binary bag-of-words vector $T_i \in \{0,1\}^{10}$, and a label $y_i \in \{0,1\}$. All 1,800 examples are produced using a local pseudorandom generator (seed 20261030). The partition is fixed at 1,260 train, 90 validation, and 450 test samples. Models are trained for exactly 12 epochs with Adam, learning rate 3×10^{-3}, mini-batch size 64, and multiclass cross-entropy. Validation is monitored but no test-dependent model selection is performed.

2.1 Why the two modalities are complementary

Two independent latent bits $a, b \in \{0,1\}$ define $y = a \oplus b$. The image encodes a via the location of a bright square, while text encodes b through a controlled word (*opacity* versus *involvement*). A separate oriented bar in the image is a noisy hint for y (correct with probability 0.83); text orientation is independently a noisy hint (correct with probability 0.76). Hence neither latent bit alone determines the target, but both together do. These probabilities are *generation parameters*, not reported empirical accuracies. This is a methodological demonstration, not a natural benchmark.

3 Model architecture

The image encoder uses convolution (1 to 8 channels, 3×3), ReLU, 2×2 max pooling, a second convolution (8 to 16 channels, 3×3), ReLU and pooling, then a 576-to-16 linear map with ReLU. The text encoder uses a 10-to-16 linear map and ReLU. Fusion concatenates the two 16-dimensional outputs, feeding a 32-to-24 ReLU layer and a 24-to-2 classifier. Unimodal baselines use the same 24-unit classifier head on their 16-dimensional embedding. Parameters are trained separately for each condition.

$$h_x = f_\theta(X), \qquad h_t = g_\phi(T), \qquad z = W_2\,\mathrm{ReLU}(W_1[h_x; h_t] + b_1) + b_2. \tag{1}$$

4 Evaluation protocol and metrics

Only the held-out 450-sample test split is used for final metrics. Accuracy is $(TP + TN)/N$; precision, recall and F_1 are macro-averaged over classes. The ablation is performed by replacing

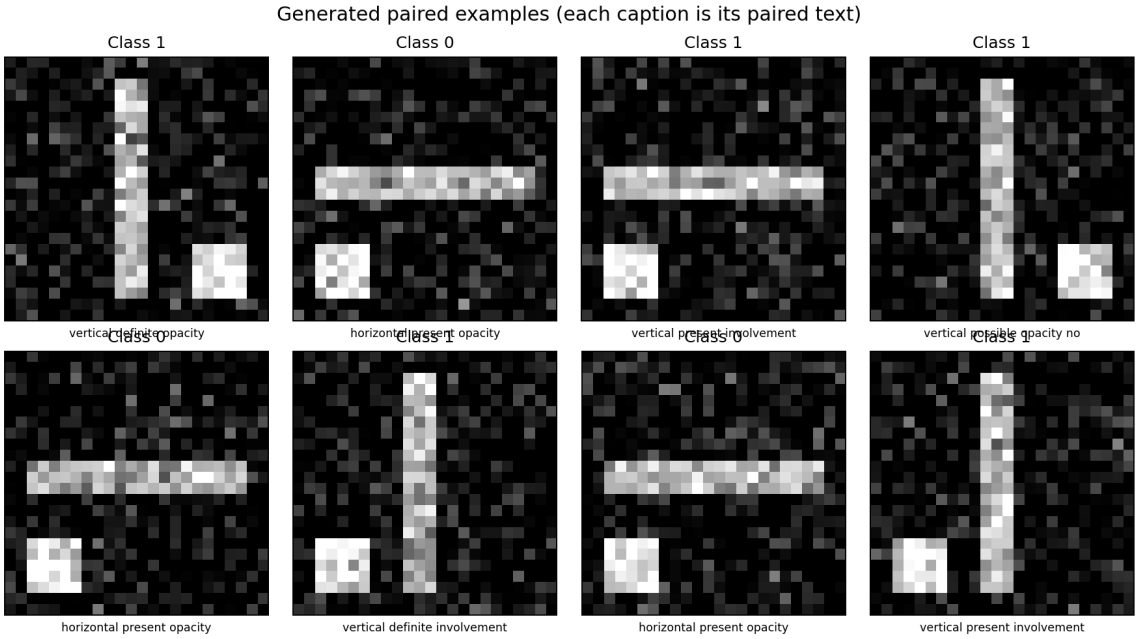


Figure 1: Actual examples sampled from the generated paired dataset. Text captions are generated from the same sample as each image.

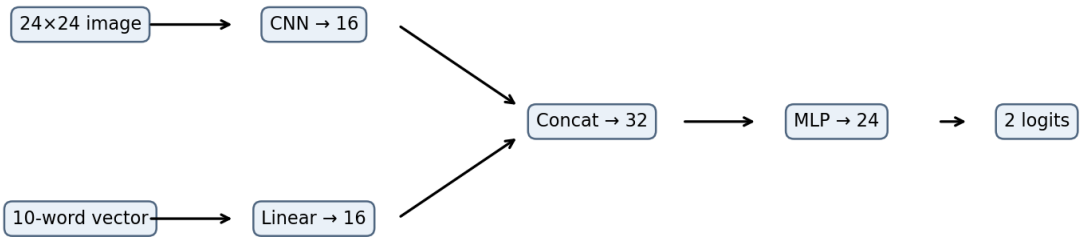


Figure 2: Implemented two-branch neural network; dimensionalities match the source code.

the learned 16-dimensional text embedding with zeros *at inference*, without retraining. This tests reliance on the trained text pathway and is not identical to retraining an image-only model.

Table 1: Measured held-out results from results/metrics.json.

Condition	Accuracy (%)	Macro F_1
Image-only	80.00	0.799
Text-only	77.11	0.770
Full fusion	100.00	1.000
Fusion, text embedding zeroed	47.78	0.475

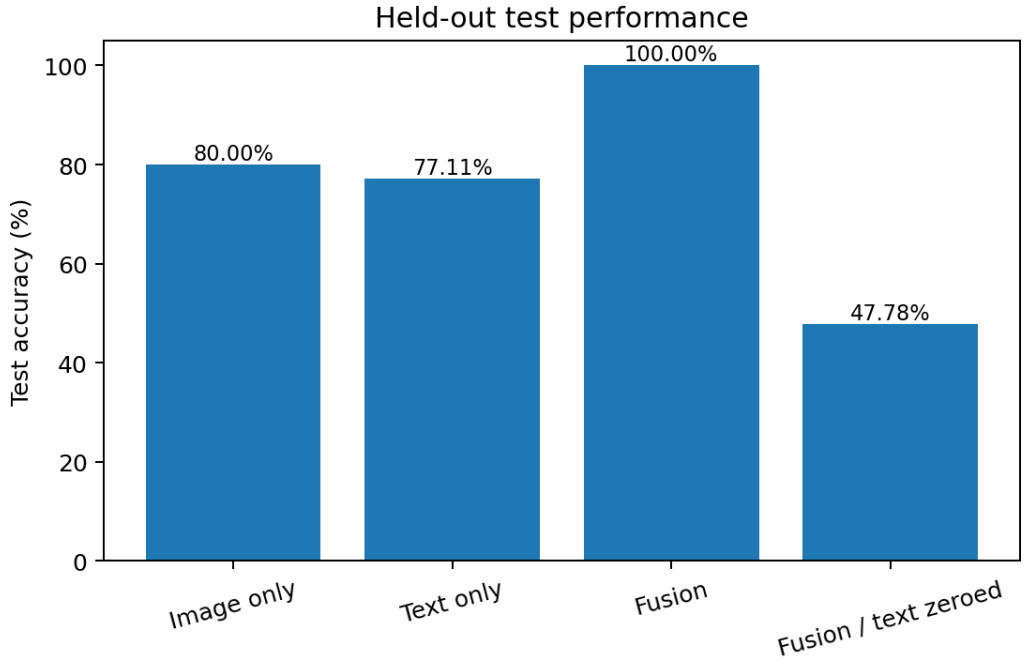


Figure 3: Held-out accuracy computed from exact test-set predictions; all four conditions use the same test rows.

5 Learning dynamics

The training trajectories below are logged epoch by epoch during the actual experiment. They are not illustrative or smoothed curves. A lower training loss does not on its own imply improved out-of-distribution robustness.

6 Confusion analysis

The trained fusion model correctly classifies 450 out of 450 held-out examples. Its confusion matrix (true labels are rows, predictions are columns) is

$$\begin{bmatrix} 207 & 0 \\ 0 & 243 \end{bmatrix}. \tag{2}$$

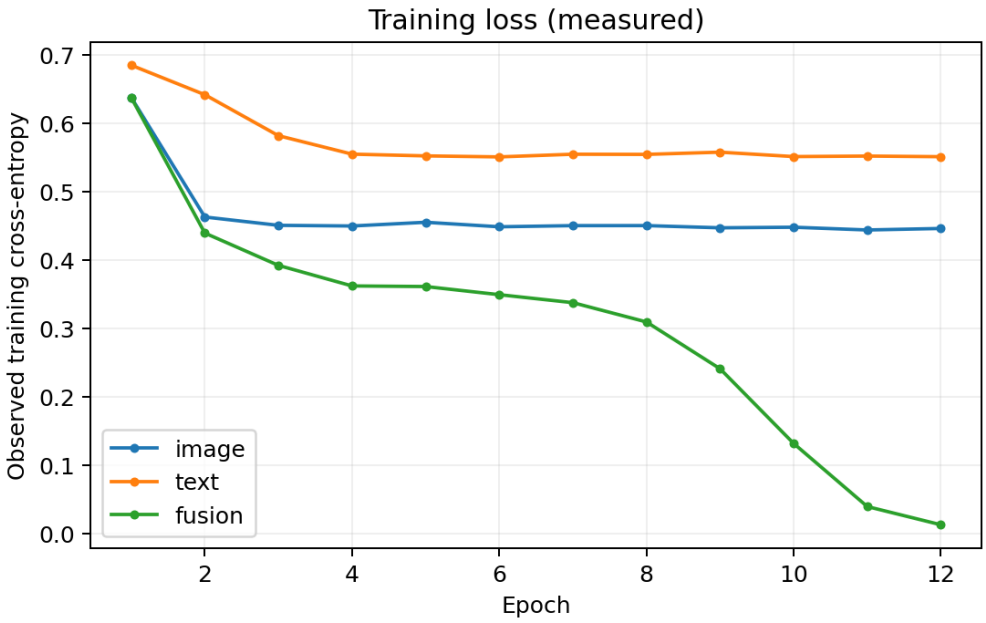


Figure 4: Observed per-epoch mean training cross-entropy for each independently trained model.

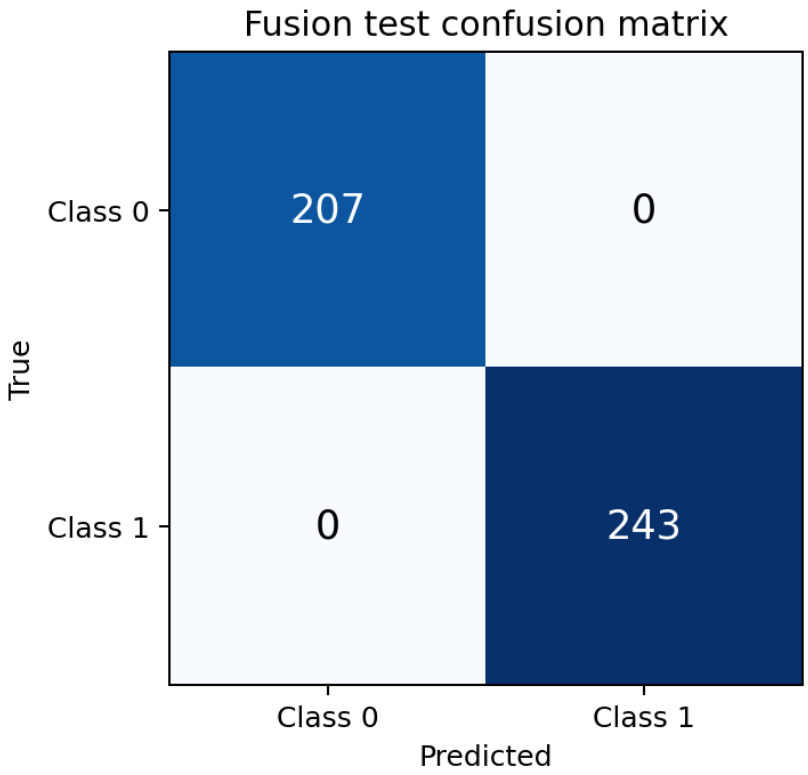


Figure 5: Fusion test predictions versus ground-truth class labels. Counts are directly calculated from saved predictions.

7 Latent representations

The 32-dimensional learned fusion embeddings of the 450 held-out samples are projected using two-component PCA. The projection is a diagnostic display, not the classifier's decision boundary, and any apparent 2D separation must not be interpreted as proof of generalization.

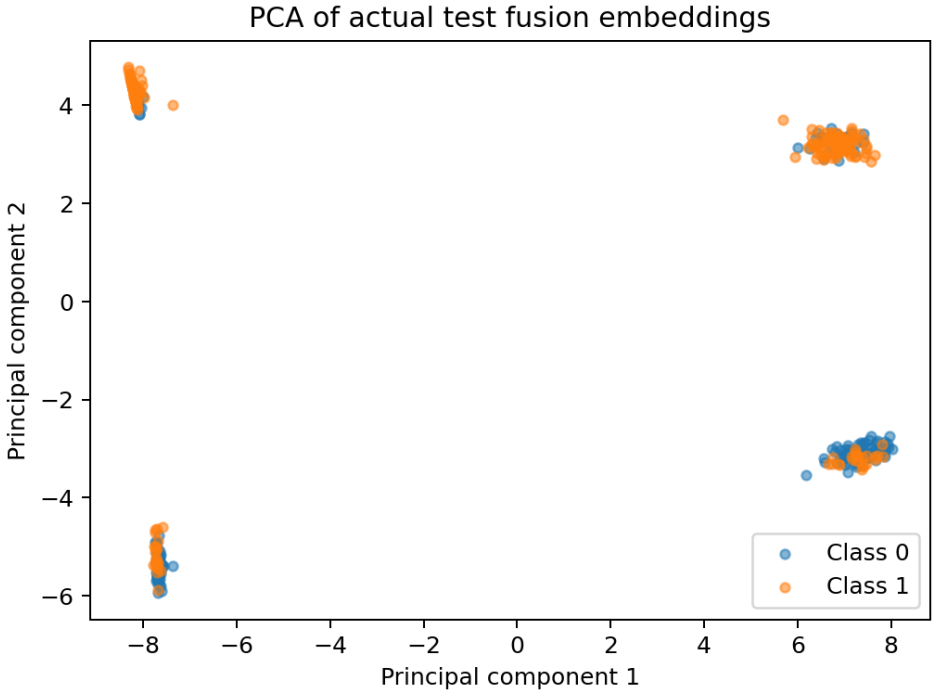


Figure 6: PCA of the measured fusion embeddings for all held-out examples.

8 Interpretation and limitations

Fusion exceeds the image-only baseline by 20.00 percentage points on this particular split. This is expected because the target contains an XOR relation across modalities: the image encodes one bit and text the other. A single held-out split and a single seed are insufficient for estimating uncertainty. Moreover, controlled keyword vectors are not general natural language; synthetic square-and-bar images are not photographs; and ablated inputs are out-of-distribution for the trained network. No external validity is claimed.

9 Reproducibility and repository contents

Run `python project.py -run` to regenerate all figures and JSON results. Run `python project.py -check` to verify stored metrics against the saved predictions and key split invariants. The report is compiled from `report.tex` using `pdflatex`. The experiment needs only Python and the pinned packages in `requirements.txt`; no network is needed once these packages are installed. The reference PDF is treated as a project specification; its example percentages and demonstrative curves are not presented as reproduced measurements.